Multimodal Email Classification: Integrating OCR with Large Language Models for Spam Detection

Aditya Chauhan, Jhanvi Nagori, Yashika, Garvit Gupta, Shubhika Sinha

May 8, 2025

Abstract

Email continues to serve as a primary mode of communication, but is persistently affected by spam. Recent advancements in Large Language Models (LLMs) present an opportunity to significantly improve spam detection performance. This study re-implements and extends the "Spam-T5" approach by evaluating RoBERTa and Flan-T5 on the publicly available datasets, and introduces an OCR-based extension for image-based email classification. This integration enhances system robustness against image-based obfuscation tactics, promoting a more resilient spam detection system.

1 Introduction

Email remains a dominant mode of communication across personal and corporate domains, but it continues to be plagued by the persistent challenge of spam. Unsolicited emails contribute significantly to digital noise, ranging from simple advertising to phishing and malware attacks. According to cybersecurity reports, a significant number of emails sent globally are spam, underscoring the urgent need for robust and adaptable spam detection systems.

Numerous machine learning models have been proposed to address this issue over the years. However, with the recent advancements in Large Language Models (LLMs), especially transformer-based architectures, the potential to detect spam more accurately and efficiently has grown significantly. These models excel at understanding semantic context and can generalize well, even in scenarios with limited labelled data.

In this project, we reimplement and extend the findings of the paper titled "Spam-T5: Benchmarking Large Language Models for Few-Shot Email Spam Detection" by Maxime Labonne and Sean Moran (JPMorgan Chase). The original paper evaluates and compares the performance of several large language models - namely RoBERTa, SetFit, and Flan-T5 - on four public datasets: Enron, Ling-Spam,SMS Spam Collection and SpamAssassin. It highlights the superiority of LLMs over traditional models, especially in few-shot learning settings, and introduces Spam-T5, a Flan-T5 model fine-tuned specifically for spam detection.

Building upon this foundation we perform the experiments using the publicly available datasets and benchmarked RoBERTa and Flan-T5 models using key metrics such as accuracy, precision, recall, and F1 score.

Additionally, we extend our approach by introducing an OCR-based spam detection, where email content embedded in images is extracted using Tesseract OCR and then classified using our RoBERTa model. This step enhances the robustness of our system against modern spam strategies that use image obfuscation to bypass traditional filters.

Through these contributions, we aim not only to validate the claims made in the JPMorgan paper but also to enhance spam detection by introducing image-text fusion capabilities. The complete implementation is available on our GitHub repository, promoting transparency and reproducibility. `https://github.com/AdityakChauhan/spam_detection`

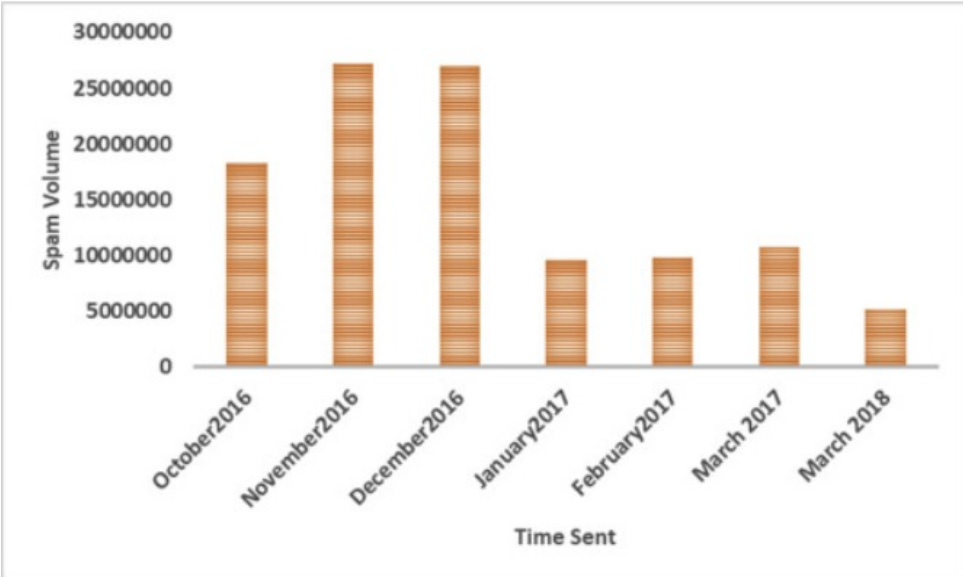

Figure 1: Volume of spam emails sent

2 Literature Review

2.1 Traditional Spam Detection Methods

Spam detection has evolved from simple rule-based systems to sophisticated machine learning techniques. Traditional methods such as Naïve Bayes, Support Vector Machines (SVMs), and Logistic Regression were widely used in early spam classification, achieving accuracy rates of 90- 95%. These models perform well with balanced data but degrade when faced with concept drift, adversarial manipulation, or limited labelled samples. Recent spam evasion tactics include embedding malicious content within images to bypass text-based filters, a challenge largely overlooked in the existing literature.

2.2 Content-Based and Deep Learning Approaches

Researchers have increasingly turned to content-based filtering using deep learning. Word embeddings (e.g., Word2Vec, GloVe) and sequence models like RNNs and LSTM networks allow better semantic understanding of email content, reaching accuracy rates of 87.1% and 83%, though they still face limitations in generalization and training efficiency.

2.3 Transformer-Based Models

Transformer-based models like BERT and RoBERTa have revolutionized natural language processing with accuracy rates exceeding 98%. In *Spam-T5: Benchmarking Large-Language Models for Few-Shot Email Spam Detection* by Labonne and Moran (2023), LLMs are evaluated on spam classification in four datasets. Key contributions include:

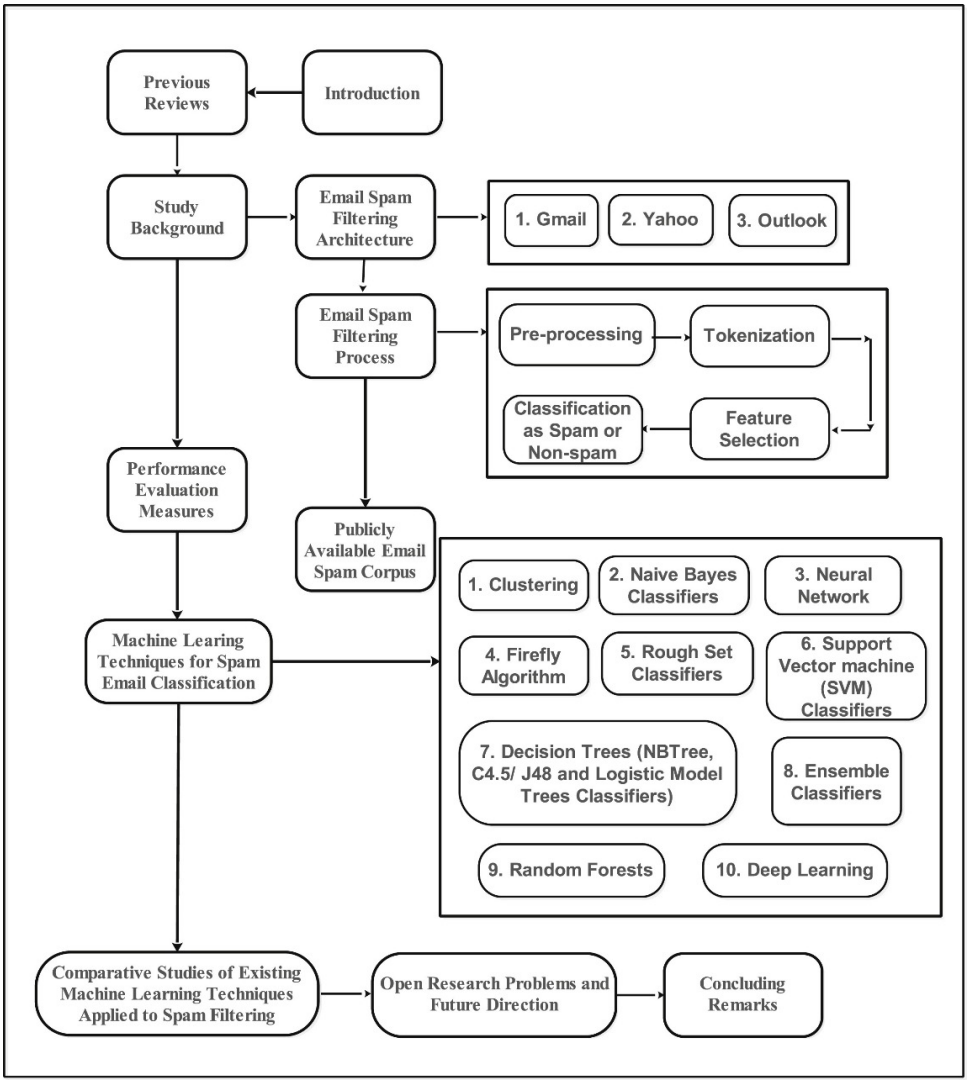

Figure 2: Overview of the email classification process using ML models

- Benchmarking LLMs against traditional baselines in full training and few-shot settings
- Introducing Spam-T5, fine-tuned specifically for email spam classification, achieving 99.1% accuracy
- Demonstrating Spam-T5's superior performance with as few as 4–16 samples
- Public release of code and models for reproducibility

The results show that LLMs significantly outperform traditional approaches, particularly when labelled data is scarce or spam patterns evolve frequently.

2.4 Classification Algorithms and Performance

2.4.1 Comparative Performance

- SVM, random forests, logistic regression, nave Bayes, and voting perceptron achieve high accuracy rates of up to 95%-95.6% in the classification of phishing emails.

- Random Forest consistently demonstrates 88.69% accuracy as reported in *A novel and secured email classification and emotion detection using hybrid deep neural network* (ScienceDirect, 2024).

2.4.2 Intelligent Classification Models

- Models using WordNet ontology and advanced text analysis techniques achieve 98.4% accuracy.
- J48 and Random Forest classifiers utilising phishing email weight calculation methods show consistent performance.

2.4.3 Multi-level Email Modelling

- Approaches that simultaneously model email headers, bodies, characters, and words reach up to 99.84% accuracy.
- NLP-based multi-level modelling proves superior to single-feature analysis methods.

3 Deep Learning Architectures for Spam Detection

Spam email detection is a sequence-based classification task. Over the years, multiple deep learning architectures have been explored to model the complex linguistic patterns and contextual semantics in emails. Here's an in-depth look at the most prominent models:

3.1 Recurrent Neural Networks (RNNs)

RNNs are the earliest form of neural networks designed for sequential data like text. They maintain a hidden state that captures information from previous tokens in a sequence, allowing them to process emails word by word while preserving order.

However, RNNs suffer from the vanishing gradient problem, their ability to retain long-term dependencies diminishes with longer sequences. Despite this, in hybrid deep neural network studies, basic RNNs have demonstrated an accuracy of **87.1%** in email classification. While decent, their performance is noticeably lower compared to more advanced models.

3.2 Convolutional Neural Networks (CNNs)

Originally developed for computer vision, CNNs have been adapted to NLP tasks due to their ability to detect local patterns. CNNs are more parallelizable and faster than RNNs but lack the ability to model long-range dependencies. In hybrid models, CNNs have achieved an accuracy of **96.%**.

3.3 Long Short-Term Memory Networks (LSTMs)

LSTMs address the limitations of basic RNNs by incorporating memory cells and gating mechanisms (input, forget, output) to better capture long-term dependencies.

They are highly effective for modeling the semantic flow in long emails, particularly those with changing tones or embedded metadata. LSTM-based classifiers have achieved **92%** accuracy in spam classification tasks.

Summary of different datasets used by other researchers for same problem.

Experiments	year	Algorithm	Dataset	Accuracy (%)
Drucker et al. [52]	1999	SVM	Emails	95
Banday and Jan [53]	2009	NB, KNN, SVM	Real life data set	95.6
DeBarr and Wechsler [54]	2009	RF	Custom collections	94.1
Shahi and Yadav [55]	2013	NB, SVM	Nepali SMS	91.64
Subramaniam et al. [56]	2010	NB	Gmail's emails	95.6
Eugene and Caswell [57]	2015	CNN	Enron	83
Abdulhamid et al. [58]	2018	ML	UCI	94.1
Du and Huang [59]	2018	LSTM	NLPCC2014 & Reuters	80.1
Lyubinets et al. [60]	2018	RNN	Arch Linux bug tracker & chromium bug tracker	87.1
Zhang, 2018 [61]	2018	CNN	Lingspam and Grumble	96
Salman et al. [50]	2022	LSTM	Lingspam	92

Figure 3: Comparison Table

3.4 Bi-directional LSTMs (Bi-LSTMs)

Bi-LSTMs process sequences in both forward and backward directions, capturing both past and future context for each token. This allows better disambiguation of contextually ambiguous words.

The DNN-BiLSTM hybrid approach achieves an accuracy of **98.69% krishnamoorthy2024hybrid**, outperforming traditional models and single-direction LSTMs.

4 Dataset

For our spam detection project, we used two publicly available datasets from Kaggle, both containing labeled email messages suitable for binary classification (spam or not spam). These datasets were chosen for their diversity and quality and were further preprocessed to match the specific needs of our machine learning models.

4.1 Dataset Sources

- **Spam Email Dataset** by Jackkson Csie **jackksonspam**: This dataset contains a total of 5,730 emails, of which 1,362 are labeled as spam and 4,368 as not spam.
- **Spam Mails Dataset** by Venky73 **venkyspam**: Derived from the Enron 1 email folder, this dataset contains 4,960 emails, including 1,392 spam and 3,568 non-spam emails.

These two datasets were merged to create a unified corpus that reflects a wide range of email content, from personal and professional communications to promotional and malicious spam.

4.2 Preprocessing of the Dataset

The following preprocessing steps were applied:

- Merged datasets and ensured label consistency.
- Removed duplicates and handled missing or corrupt entries.
- Retained only essential columns such as email text and labels.
- Standardized the format of the email text for consistent feature extraction.

4.3 Final Preprocessed Dataset Statistics

Dataset	**Total Emails**	**Spam**	**Ham**	**Spam Ratio**
Spam Email Dataset(by jackksonspam	5,730	1,362	4,368	∼24%
Enron 1	4,960	1,392	3,568	∼28%
Combined Total	**10,690**	**2,754**	**7,936**	∼26%

Table 1: Spam-Ham Distribution across Individual dataset

4.4 OCR Test Samples

In addition to textual datasets, we curated a small test batch of **spam screenshots** containing embedded text, such as promotional banners. These images were processed using our OCR system based on `pytesseract` and included in the model's inference testing phase.

- Images include screenshots of spam emails.
- Text was extracted using the OCR and passed to the fine-tuned RoBERTa classifier.
- Although this dataset was **not used during training**, it provides a benchmark for evaluating the **OCR-augmented model** in real-world, image-based scenarios.

This setup allows us to test how well our system generalizes to spam content presented in visual form, often used to bypass traditional spam filters.

5 Methodology

This section elaborates on the technical steps taken to implement and fine-tune RoBERTa and Flan-T5 for binary email spam detection. Both models were trained using Hugging Face's 'transformers' library with PyTorch backend, and evaluated using standard classification metrics. A modular structure was followed to support easy integration of future components, such as OCR-based detection.

5.1 Overview

a) **Data Collection:** The datasets employed in this study were sourced from Kaggle repositories: the Spam Email Dataset by Jackkson Csie and the Spam Mails Dataset by Venky73. These datasets comprise a combined total of 10,690 emails, with varying distributions of spam and non-spam messages, offering a comprehensive foundation for our analysis.

b) **Preprocessing:** Both datasets have already handled HTML tags and other metadata embedded within the text. The datasets were concatenated, missing values were removed, and duplicate entries were identified and eliminated to ensure data integrity.

c) **Tokenization:** Each model used its corresponding Hugging Face tokenizer (e.g., RobertaTokenizer for RoBERTa). Tokenization included padding to max length of 128 for RoBERTa and 256 for FlanT5, truncation, and conversion to tensor format.

d) **Fine-Tuning:** Fine-Tuning: A pre-trained RoBERTa model was fine-tuned for the binary spam classification task using a custom training loop implemented in PyTorch. The AdamW optimizer with a learning rate of 2e-5(RoBERTa) and 5e-5(FlanT5) was used for optimisation.

e) **Evaluation:** Evaluation was performed using Accuracy, Precision, Recall, and F1-Score. The model with the highest validation F1-Score was saved and used for final testing.

5.2 RoBERTa

We used the pre-trained `roberta-base` model from Hugging Face, which has 125 million parameters and is designed for high-performance natural language understanding tasks.

5.2.1 Preprocessing and Tokenization:

Each email message was converted to a string and missing values in the label column were dropped. The text was already cleaned of html headers, url, etc in the dataset.

Tokenization was handled using the `AutoTokenizer` from Hugging Face(Roberta Tokenizer) with the `roberta-base` model. Texts were padded and truncated to a maximum length of 128 tokens. The following code snippet illustrates this process:

Listing 1: Tokenization with AutoTokenizer

```
from transformers import AutoTokenizer

tokenizer = AutoTokenizer.from_pretrained(roberta-base)
inputs = tokenizer(
    texts,
    padding=max_length,
    truncation=True,
    max_length=128,
    return_tensors
)
```

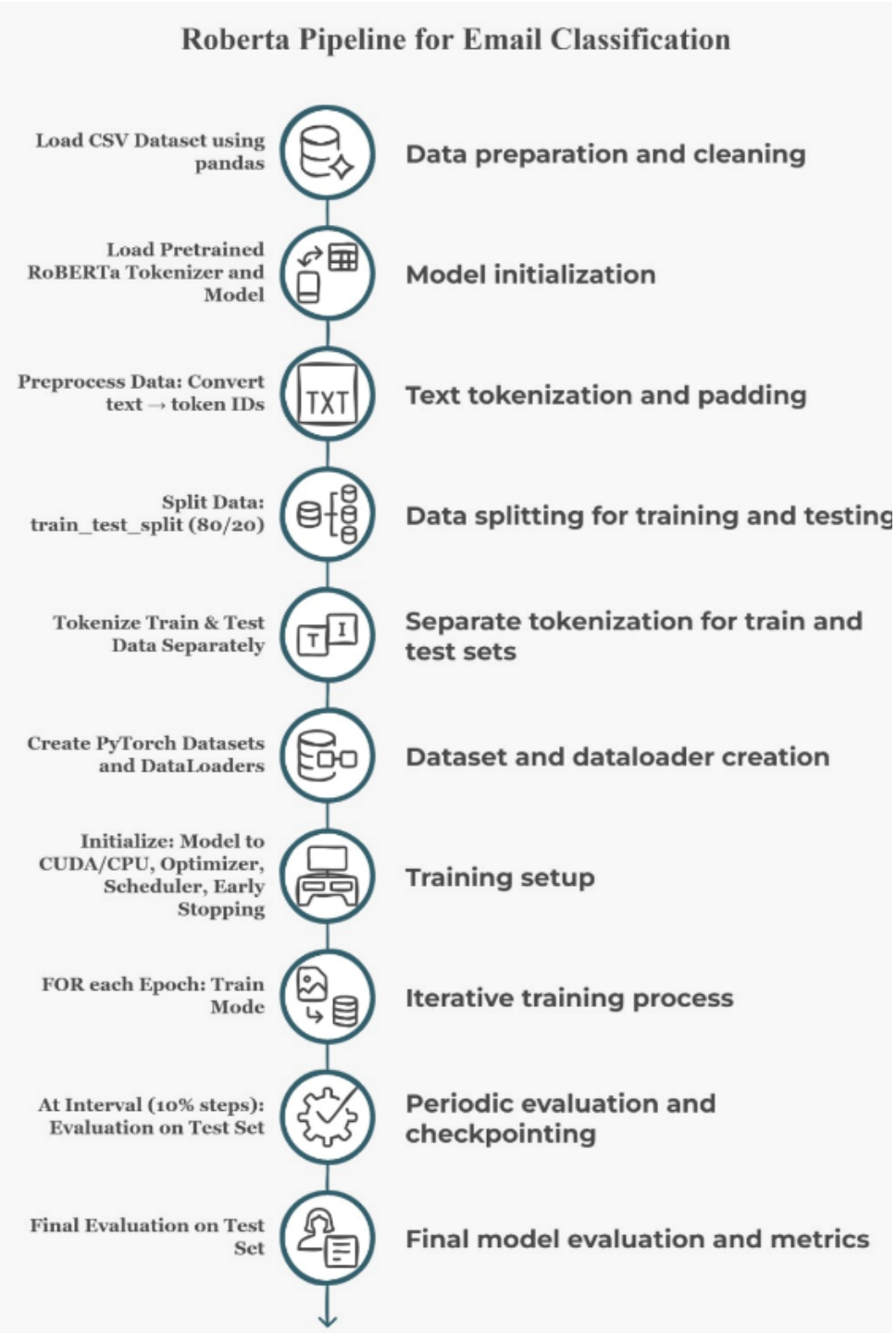

Figure 4: The RoBERTa Pipeline for Email Classification

5.2.2 Model Setup and Training

We used `AutoModelForSequenceClassification` from the Hugging Face Transformers library, initialized with the `roberta-base` checkpoint. The model was configured for binary classification (spam vs ham).

Training details were as follows:

- **Loss Function:** Cross-Entropy Loss

- Formula for Cross-Entropy Loss:

$$L(y, \hat{y}) = -\frac{1}{N} \sum_{i=1}^{N} \sum_{c=1}^{C} y_{ic} \log(\hat{y}_{ic})$$

- **Optimizer:** AdamW
  AdamW is a variant of the Adam optimizer that decouples weight decay from the optimization steps. Adam (short for Adaptive Moment Estimation) is a popular optimization algorithm in deep learning, combining the benefits of both Adagrad and RMSprop. It maintains adaptive learning rates for each parameter by computing first and second moments of the gradients. Specifically, Adam uses the following updates:

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t$$
$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2$$
$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1 - \beta_2^t}$$
$$\theta_t = \theta_{t-1} - \frac{\alpha}{\sqrt{\hat{v}_t} + \epsilon} \hat{m}_t$$

  Where: - β_1 and β_2 are the decay rates for the moving averages of the gradient and its square (typically set to 0.9 and 0.999, respectively). - g_t is the gradient at time step t. - m_t and v_t are the first and second moment estimates of the gradients. - $\hat{m}_t$ and $\hat{v}_t$ are the bias-corrected moment estimates. - α is the learning rate, and ϵ is a small constant added to prevent division by zero.

  The Adam optimizer is highly effective for sparse gradients, noisy data, and varying learning rates across parameters. The inclusion of weight decay in AdamW (which is simply adding a regularization term) further helps with generalization in some cases.

- **Learning Rate:** 2×10^{-5}

- **Evaluation:** Greedy decoding followed by string matching on normalized predictions and labels.

- **Token Sequence Length:** 128

- **Early Stopping:** Implemented based on validation F1 score.

The following code snippet illustrates the model and optimizer setup:

Listing 2: Initializing RoBERTa model and AdamW optimizer

```
from transformers import RobertaForSequenceClassification,
    AdamW

# Load pre-trained RoBERTa model with two output labels (spam
    and ham)
model = RobertaForSequenceClassification.from_pretrained("
    roberta-base", num_labels=2)

# Define the optimizer with learning rate 2e-5
optimizer = AdamW(model.parameters(), lr=2e-5)
```

5.2.3 Training

Training was implemented using PyTorch to fine-tune the RoBERTa model. The training process involved iterating over the training data in batches loaded by the `DataLoader`. For each batch, the following steps were performed:

1. A forward pass of the input tensors (input IDs and attention masks) through the model to obtain the output logits.
2. Calculation of the loss using the Cross-Entropy Loss function, which is implicitly handled by the model when labels are provided.
3. Backward propagation of the loss to compute gradients of the model's parameters.
4. Optimization of the model's parameters using the AdamW optimizer with a learning rate of 2×10^{-5}.

The training progress was monitored by evaluating the F1 score on the test set at regular intervals during training. These metrics, along with the training loss, were visualized using live plotting to provide real-time feedback on model performance.

5.2.4 Evaluation

After training, the model was evaluated on the test set. The evaluation involved:

- Predicting class labels using:

```
torch.argmax(logits, dim=1)
```

- Computing the following metrics:
  - **Accuracy**
  - **Precision**
  - **Recall**
  - **F1-Score** (both macro and binary averaged)

5.3 Flan-T5

Flan-T5 treats classification as a text generation task, leveraging a pre-trained encoder-decoder transformer. This architecture allows it to approach classification problems using natural language prompts.

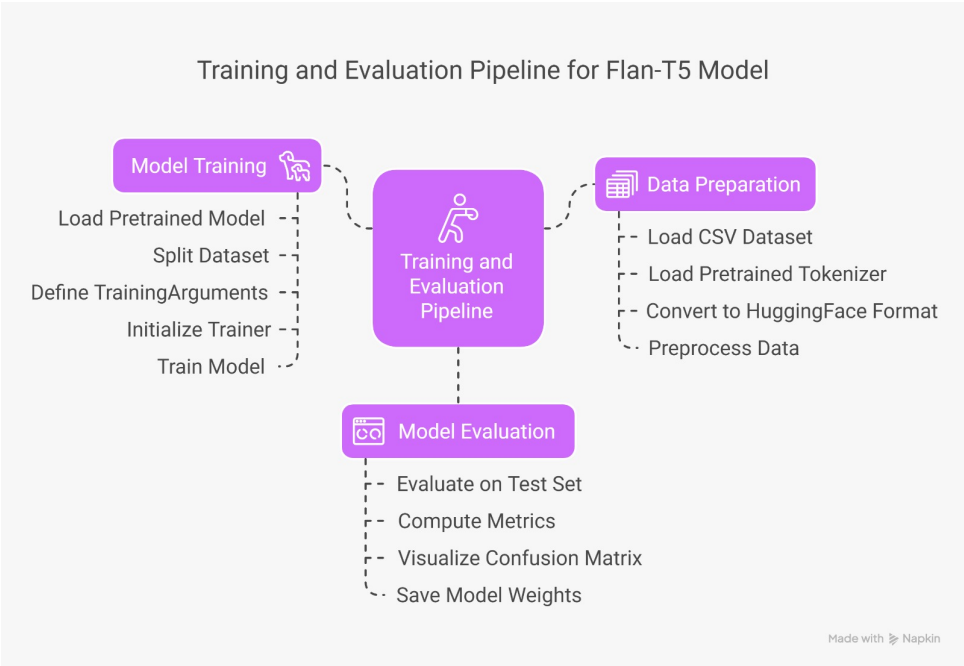

Figure 5: Training and Evaluation Pipeline for Flan-T5 Model

5.3.1 Input Format

Each email sample was transformed into a natural language prompt format before being fed into the model. The input structure followed the format:

```
classify:  <email content>
```

This format enables the model to use its instruction-following and natural language understanding capabilities to perform classification.

5.3.2 Model Configuration

The configuration used for the spam classification task was as follows:

- **Model:** `google/flan-t5-small`, a fine-tuned version of T5 with approximately 80 million parameters.
- **Tokenizer:** `T5Tokenizer`, based on SentencePiece.
- **Target Labels:** `"spam"` and `"ham"`.
- **Preprocessing:** Padding and truncation were applied to ensure the input length did not exceed 256 tokens.

5.3.3 Model Initialization

The model and tokenizer were loaded using the Hugging Face `transformers` library. Flan-T5 is suitable for conditional generation tasks, including sequence-to-sequence classification.

Listing 3: Model and Tokenizer Initialization

```
from transformers import T5Tokenizer,
    T5ForConditionalGeneration

tokenizer = T5Tokenizer.from_pretrained('google/flan-t5-small'
    )
model = T5ForConditionalGeneration.from_pretrained('google/
    flan-t5-small')
```

5.3.4 Data Preparation

The numeric labels (0 for ham, 1 for spam) were converted to their string representation. Both the prompted input text and the string labels were tokenized using the T5Tokenizer with padding to a maximum length and truncation.

- **Max Input Length:** 256 tokens
- **Max Target Length:** 10 tokens

```
input_text = "classify:" + example["text"]
target_text = str(example["label"])

input_enc = tokenizer(
    input_text,
    padding=max_length,
    truncation=True,
    max_length=256
)

target_enc = tokenizer(
    target_text,
    padding=max_length,
    truncation=True,
    max_length=10
)

labels = target_enc["input_ids"]
labels = [l if l != tokenizer.pad_token_id else -100 for l in labels]
```

5.3.5 Training Setup

The model was fine-tuned using a sequence-to-sequence loss over the target tokens. The training configuration included:

- **Loss Function:** Cross-entropy loss over the generated token sequence:

$$L(y, \hat{y}) = -\frac{1}{N}\sum_{i=1}^{N}\sum_{c=1}^{C} y_{ic}\log(\hat{y}_{ic})$$

- **Optimizer:** AdamW
- **Learning Rate:** 5×10^{-5}
- **Evaluation:** Greedy decoding followed by string matching on normalized predictions and labels.

```
outputs = model.generate(input_ids=input_ids, attention_mask=
    attention_mask, max_new_tokens=10)
preds.extend(tokenizer.batch_decode(outputs,
    skip_special_tokens=True))
```

5.4 OCR Integration for Image-Based Email Classification

To enhance our spam detection capabilities beyond plain-text emails, we implemented an OCR-based pipeline that enables the classification of image-based emails and screenshots—an increasingly common tactic to bypass traditional spam filters. This system leverages OpenCV for image preprocessing, `pytesseract` for Optical Character Recognition, and a fine-tuned RoBERTa model for text classification, allowing us to evaluate the performance of our RoBERTa-based spam classifier on image-based email content.

Pipeline Steps

- Load the input image using `cv2.imread`.
- Preprocess the image by converting it to grayscale to improve OCR accuracy.
- Apply `pytesseract.image_to_string` to extract textual content from the image.
- Pass the extracted text through the fine-tuned RoBERTa model for classification.
- Output the prediction label (`Spam` or `Ham`) along with the associated class probabilities.

Listing 4: OCR Pipeline for Spam Detection

```
gray_image = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
ocr_text = pytesseract.image_to_string(gray_image)

result = predict_spam(ocr_text)[0]
```

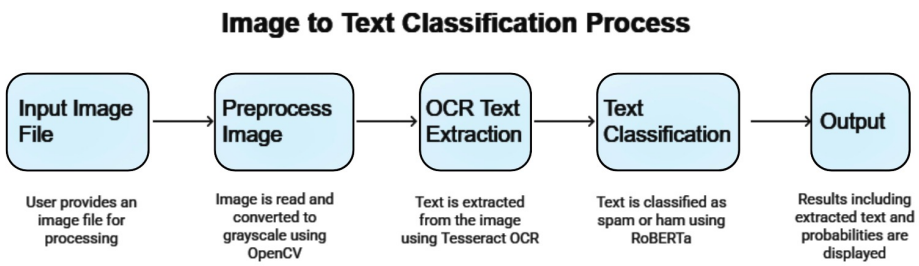

Figure 6: Pipeline for OCR based classification

6 Experiments

This section describes the experimental setup used to evaluate and compare the performance of **RoBERTa** and **Flan-T5** models for spam classification.

6.1 Training Configuration

Both models were fine-tuned on the publicly available datasets using a stratified 80/20 train-test split to maintain label balance across sets.

Environment Setup

- **Framework:** PyTorch with Hugging Face Transformers

6.2 Evaluation Metrics

To evaluate model performance, we computed the following metrics on the test set:

- **Accuracy**: Overall correctness of predictions.
- **Precision**: Correct spam predictions out of all predicted spam.
- **Recall**: Correct spam predictions out of all actual spam.
- **F1 Score**: Harmonic mean of precision and recall.

Additionally, we plotted the **confusion matrix** to visually inspect false positives and false negatives.

6.3 Results for Flan-T5

Metric	Value
Accuracy	**0.9846**
Precision	**0.9702**
Recall	**0.9719**
F1 Score	**0.9710**

Table 2: Evaluation Metrics for Flan-T5 on Test Set

Confusion Matrix:

	Predicted Ham	Predicted Spam
Actual Ham	1552	17
Actual Spam	16	553

Table 3: Confusion Matrix for Flan-T5

- Very low false positive and false negative rates.

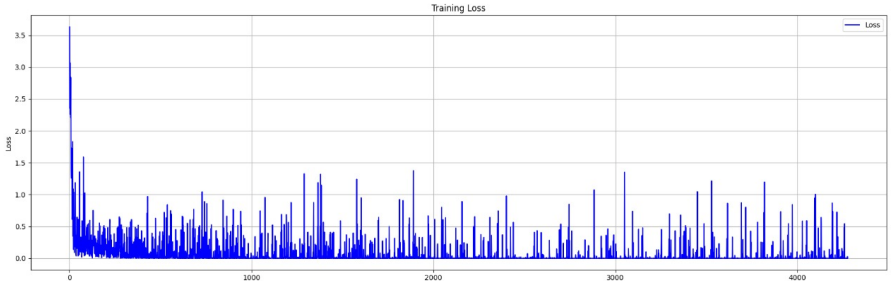

Figure 7: Training Progress of Flan-T5

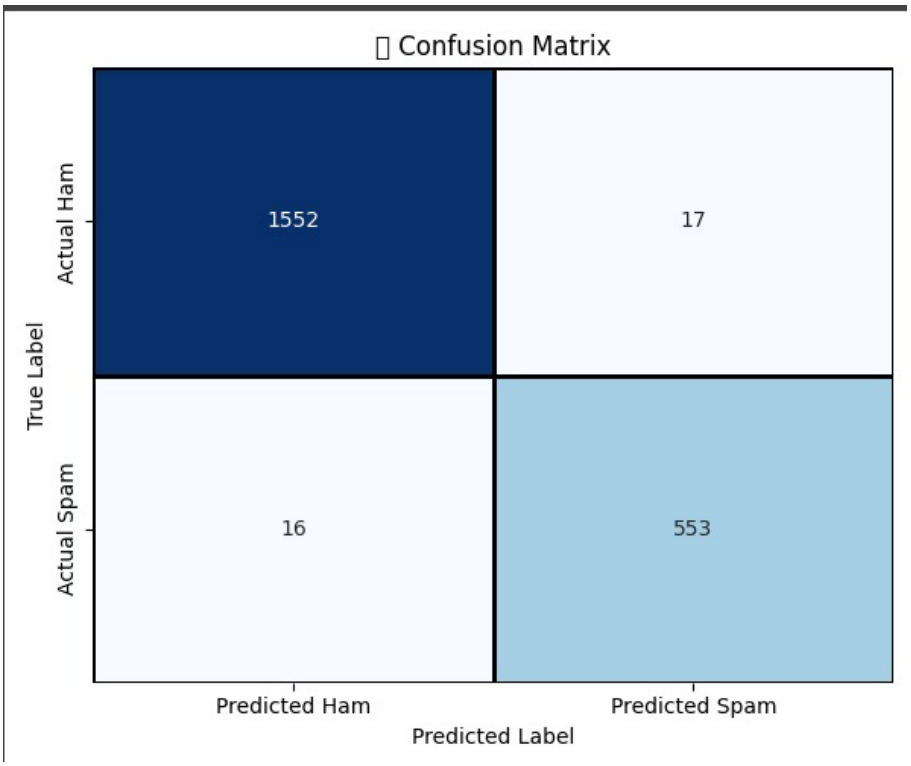

Figure 8: Confusion Matrix of Flan-T5

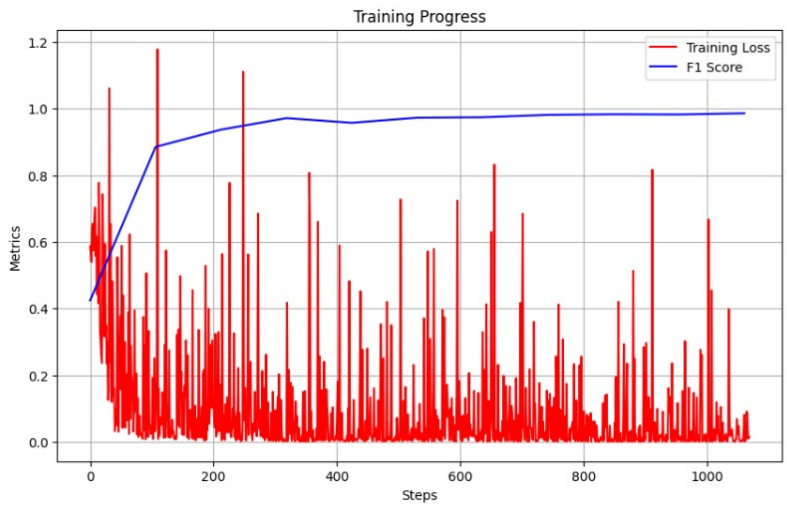

Figure 9: Training Progress of RoBERTa

6.4 Results for RoBERTa

The results for RoBERTa, based on experimental evaluation:

Metric	Value
Accuracy	**0.99**
Precision	**0.99**
Recall	**0.98**
F1 Score	**0.9860**

Table 4: Evaluation Metrics for RoBERTa on Test Set

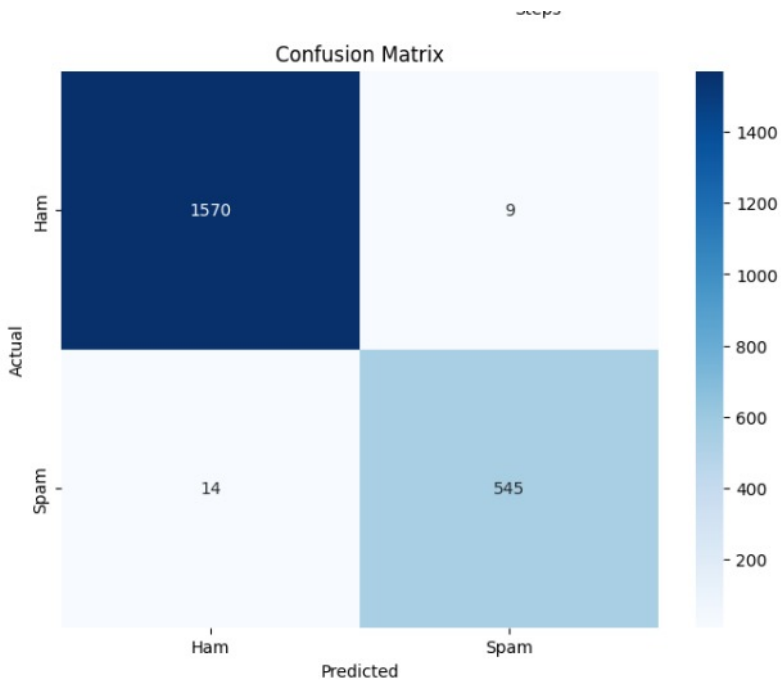

Figure 10: Confusion Matrix of RoBERTa

- Slightly better F1 and accuracy than Flan-T5.
- Selected as the model for future OCR integration due to superior accuracy.

7 Results and Discussion

The experimental results reveal important insights into the relative strengths and weaknesses of the RoBERTa and Flan-T5 models for email spam detection.

7.1 Comparative Performance

Model	Accuracy	Precision	Recall	F1 Score
RoBERTa	0.99	0.99	0.98	**0.9860**
Flan-T5	0.9846	0.9702	0.9719	**0.9710**

Table 5: Performance Comparison of RoBERTa vs. Flan-T5

RoBERTa slightly outperforms Flan-T5 in terms of accuracy and F1 score. This aligns with

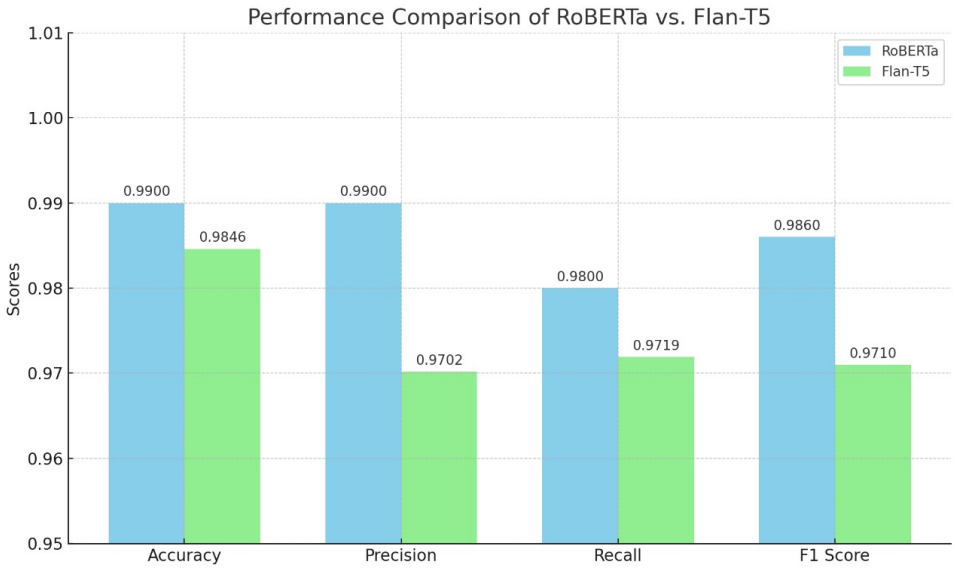

Figure 11: Performance comparison of RoBERTa and Flan-T5

RoBERTa's architecture, which is optimized for classification tasks. Its bidirectional transformer allows for deep contextual understanding, particularly effective for structured emails.
Flan-T5, while slightly behind on full-data metrics, demonstrated superior few-shot performance. Its instruction-tuned design enables effective generalization from limited data, making it highly suitable for rapidly adapting to evolving spam tactics or low-resource settings.

7.2 Implications for Real-World Deployment

Both models delivered great results in spam classification. However, their ideal deployment contexts might differ:

- **RoBERTa** is better suited for server-side or batch processing applications, where computational resources are available and high accuracy is a priority.
- **Flan-T5** is more suitable for client-side or mobile environments, where adaptability and few-shot learning capabilities are advantageous.

Given these findings, we selected **RoBERTa for integration with OCR-based spam detection** due to its superior accuracy and robustness in handling noisy, image-extracted textual data.

Preliminary results show that, despite the RoBERTa model being originally trained on clean, non-OCR text, it maintains a stable ability to distinguish between spam and ham when applied to OCR-extracted text. This suggests a promising level of adaptability and robustness in handling real-world formats where emails are embedded within images to evade traditional spam filters.

8 Conclusion

This study explored the application of **Large Language Models (LLMs)** in the domain of email spam detection, focusing on two state-of-the-art architectures: **RoBERTa** and **Flan-T5**. By re-implementing the methodology outlined in the JPMorgan Chase paper *"Spam-T5: Benchmarking Large Language Models for Few-Shot Email Spam Detection"*, we expanded the scope by integrating real-world enhancements, including OCR-based email image classification.

Our experiments demonstrate that both models achieve **high classification accuracy**, with RoBERTa slightly outperforming Flan-T5 in full-data training scenarios, while Flan-T5 excels in low-resource, few-shot learning setups.

To address modern spam tactics involving image obfuscation, we implemented an OCR-based image classification pipeline. When coupled with RoBERTa, the system maintained strong classification accuracy even on noisy, OCR-extracted text, demonstrating its robustness and readiness for adversarial real-world deployments.

However, it is important to note that the model's performance is highly dependent on the quality and diversity of the training data. Biases or gaps in the training set can significantly affect the classifier's ability to generalize to unseen or adversarial spam formats.

By combining textual and image-based spam detection, this project offers a more holistic approach to intelligent spam filtering. It showcases the viability of LLMs in production environments and highlights the importance of adaptability in security-focused applications.

However, challenges persist with image-based spam. Spammers often employ obfuscation techniques, such as distorted fonts, noisy backgrounds, or compression artifacts, to reduce OCR accuracy. Consequently, the overall performance of the spam detection system depends heavily on the quality and clarity of the OCR output. Improving OCR preprocessing and handling noisy text extraction remains an important direction for future enhancement.

Key contributions of this work include:

- Benchmarking modern LLMs on structured datasets
- Integrating OCR-based email image classification

Future work may focus on:

- Introducing adversarial training to further enhance model resilience

- Improving OCR preprocessing techniques to better handle heavily obfuscated image spam
- Expanding and diversifying the training dataset to strengthen model generalization and reduce bias

References

[1] M. Labonne and S. Moran, "Spam-T5: Benchmarking Large Language Models for Few-Shot Email Spam Detection," *arXiv preprint arXiv:2304.01238*, 2023. [Online]. Available: https://arxiv.org/abs/2304.01238

[2] P. Krishnamoorthy, M. Sathiyanarayanan, and H. P. Proença, "A novel and secured email classification and emotion detection using a hybrid deep neural network," *International Journal of Cognitive Computing in Engineering*, vol. 5, pp. 44–57, 2024. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S2666307424000019

[3] P. Charanarur, H. Jain, G. S. Rao, D. Samanta, S. S. Sengar, and C. T. Hewage, "Machine-Learning-Based Spam Mail Detector," *SN Computer Science*, vol. 4, no. 1, p. 858, 2023. [Online]. Available: https://link.springer.com/article/10.1007/s42979-023-02330-x

[4] M. A. Shaaban, Y. F. Hassan, and S. K. Guirguis, "Deep convolutional forest: a dynamic deep ensemble approach for spam detection in text," *Complex & Intelligent Systems*, vol. 8, pp. 4897–4909, 2022. [Online]. Available: https://link.springer.com/article/10.1007/s40747-022-00741-6

[5] K. I. Roumeliotis, N. D. Tselikas, and D. K. Nasiopoulos, "Next-Generation Spam Filtering: Comparative Fine-Tuning of LLMs, NLPs, and CNN Models for Email Spam Classification," *Electronics*, vol. 13, no. 11, p. 2034, 2024. [Online]. Available: https://doi.org/10.3390/electronics13112034

[6] A. Sharma, R. Kumar, and S. Verma, "Email Spam Classification Based on Deep Learning Methods: A Review," *Journal of Information Security and Applications*, vol. 72, p. 103456, 2024. [Online]. Available: https://doi.org/10.1016/j.jisa.2024.103456

[7] R. Singh, M. Kaur, and P. Sharma, "A hybrid correlation-based deep learning model for email spam classification using fuzzy inference system," *Journal of King Saud University - Computer and Information Sciences*, vol. 35, no. 10, pp. 1234–1245, 2023. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S2772662223002308

[8] E. G. Dada, J. S. Bassi, H. Chiroma, S. M. Abdulhamid, A. O. Adetunmbi, and O. E. Ajibuwa, "Machine learning for email spam filtering: review, approaches and open research problems," *Heliyon*, vol. 5, no. 6, p. e01802, 2019. [Online]. Available: https://doi.org/10.1016/j.heliyon.2019.e01802

[9] Jackksoncsie "Spam Email Dataset," *Kaggle*, 2023. [Online]. Available: https://www.kaggle.com/datasets/jackksoncsie/spam-email-dataset?resource=download

[10] Venkatesh Garnepudi

"Spam Mails Dataset," *Kaggle*, 2023. [Online]. Available: https://www.kaggle.com/datasets/venky73/spam-mails-dataset